DinisCruz added more briefs for 09 mar

d4bb25b · now

783 lines (663 loc) · 47.9 KB

Preview Code Blame Raw

SG/Send Architecture — How It Actually Works

Version: v0.15.5
From: Dinis + the SG/Send team
Date: March 2026
Purpose: Three architecture snapshots — today, near-future PKI, and secure pods

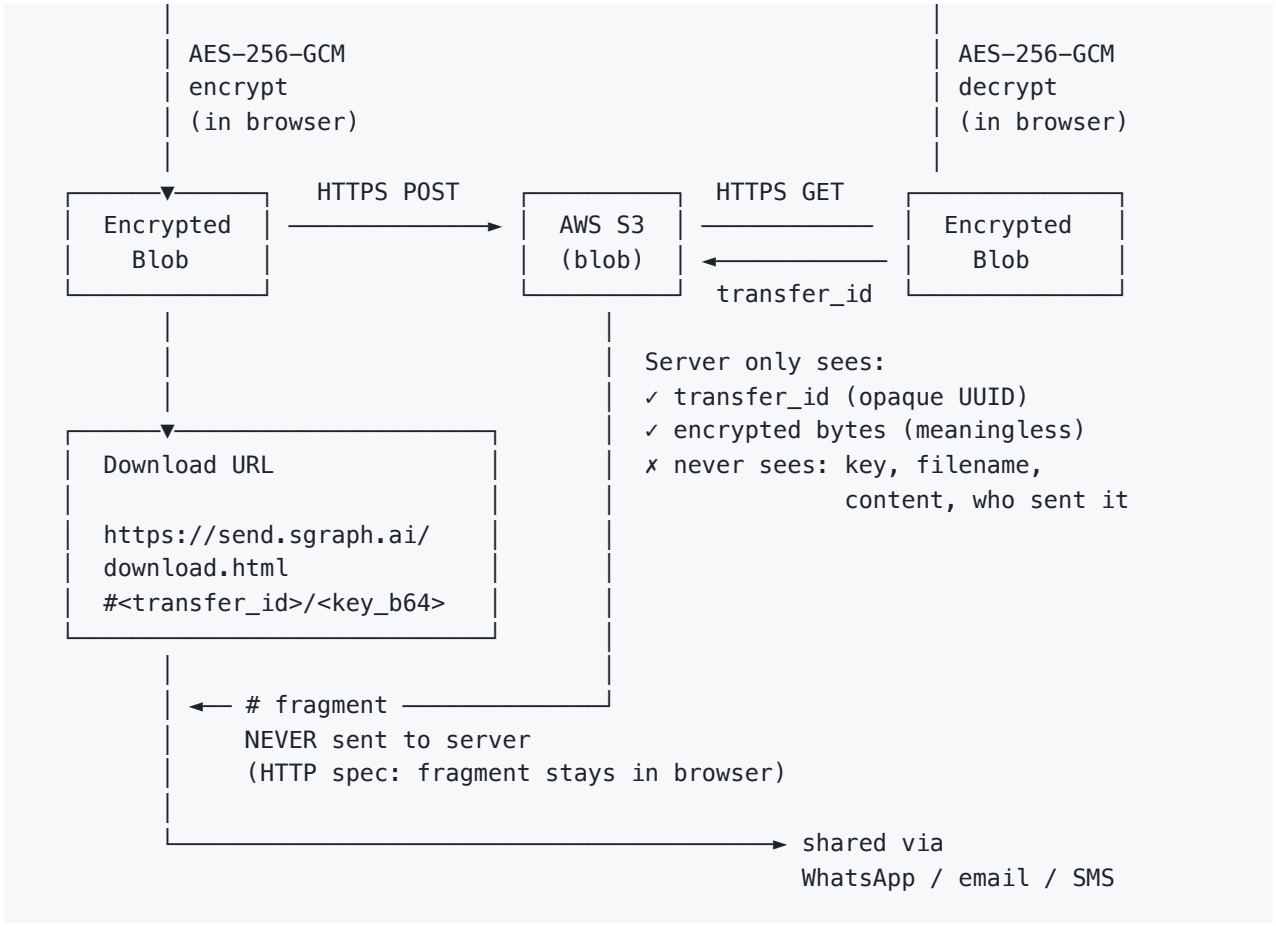
> **The one-sentence principle that drives every decision: The server must never be able to read the file.** Not now, not if subpoenaed, not if breached. Everything else follows from that.

SECTION 1 — What Exists Today

The Core Model

The browser is the encryption engine. The server is a dumb blob store. The key never touches the server — ever.

```
┌──────────────────────────────────────────────────────────────┐
│                      SG/SEND TODAY                           │
│                Zero-Knowledge Symmetric Model                │
└──────────────────────────────────────────────────────────────┘

SENDER SIDE (browser)          SERVER          RECIPIENT SIDE (browser)
─────────────────────          ──────          ────────────────────────

┌──────────┐                                   ┌──────────┐
│ Your File │                                   │ Your File │
│ (plaintext)│                                  │ (plaintext)│
└─────┬────┘                                   └────▲─────┘
```



Encryption Flow — Step by Step

```
SENDER BROWSER
═══════════════════════════════════════════════════════════════════

  1. WRAP                          2. GENERATE KEY
  ──────────────────────────       ──────────────────────────
  plaintext file bytes              crypto.getRandomValues()
         +                                  │
  SGMETA envelope                           ▼
  {"filename": "report.pdf"}        32-byte AES key  ← lives ONLY in browser
memory
          │                                 │
          ▼                                 │
  ┌──────────────────────────┐              │
  │ SGMETA\x00[4b][json]     │              │
  │ [file bytes...........]  │              │
  └──────────────────────────┘              │
                │                           │
                └─────────────┬─────────────┘
                              │
                              ▼
  3. ENCRYPT
  ──────────────────────────
  AES-256-GCM
  input:   wrapped payload
  key:     random 32 bytes
  nonce:   random 12 bytes (GCM)
           │
           ▼
```

```
  ┌──────────────────────────────────────────────────────┐
  │ [12-byte nonce][ciphertext][16-byte GCM auth tag]    │
  └──────────────────────────────────────────────────────┘

           │
           ▼
  base64 encode → upload to S3

  4. BUILD URL
  ──────────────────────────
  key_b64url = base64url_nopad(key_bytes)

  https://send.sgraph.ai/download.html#<transfer_id>/<key_b64url>
                                       ▲
                                       └─ this part is NEVER in HTTP request
                                          browsers don't send fragments to
servers

═══════════════════════════════════════════════════════════════════════
RECIPIENT BROWSER
═══════════════════════════════════════════════════════════════════════

  URL arrives → browser parses fragment locally (no server call)
       │
       ▼
  extract transfer_id → GET /transfers/download/{transfer_id}
       │                     (server returns encrypted blob)
       ▼
  extract key_b64url  → decode → 32-byte AES key
       │
       ▼
  AES-256-GCM decrypt (GCM auth tag checked — any tampering = instant fail)
       │
       ▼
  strip SGMETA envelope → recover original filename + file bytes
       │
       ▼
  browser download prompt
       │
       ▼
  history.replaceState() → wipe key from URL bar + browser history entry
```

What the Server Actually Stores

```
┌──────────────────────────────────────────────────┐
│  SG/SEND SERVER DATABASE — what's actually in it │
├──────────────────────┬───────────────────────────┤
│  Field               │  Example value            │
├──────────────────────┼───────────────────────────┤
│  transfer_id         │  a3f7b9c2-...  (UUID)     │
│  status              │  "completed"              │
│  file_size_bytes     │  248832                   │
│  content_type_hint   │  "application/pdf"        │
│  created_at          │  2026-03-09T14:22:00Z     │
│  access_token_ref    │  "sg-send__dinis__abc1"   │
│  download_count      │  1                        │
├──────────────────────┼───────────────────────────┤
│  WHAT IS NOT STORED  │                           │
```

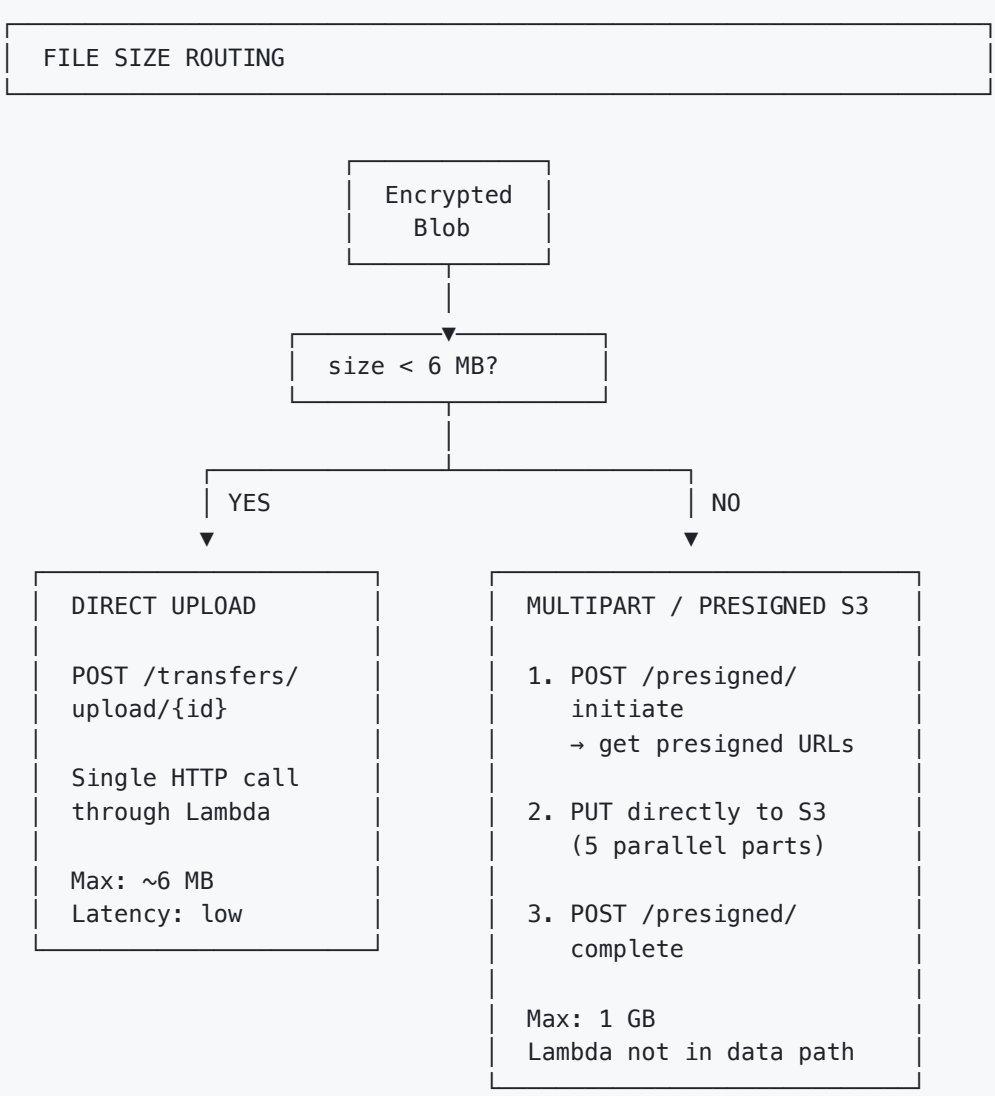
```
┌──────────────────────────┬──────────────────────────────┐
│  encryption key          │  ✗ — never written           │
│  original filename       │  ✗ — inside encrypted blob   │
│  file content            │  ✗ — encrypted in S3         │
│  sender identity         │  ✗ — token is opaque         │
│  recipient identity      │  ✗ — no recipient concept    │
└──────────────────────────┴──────────────────────────────┘
```

The replaceState Situation

ReplaceState doesn't "erase history" in the traditional sense. Here's exactly what it does and doesn't do:

```
TIMELINE OF A DOWNLOAD SESSION
═══════════════════════════════════════════════════════════════

  t=0   User navigates to:
        https://send.sgraph.ai/download.html#abc123/xK9mP2secretkey

        Browser history stack:
        ┌──────────────────────────────────────────┐
        │  [prev page]                             │  ← index 0
        │  [prev page 2]                           │  ← index 1
        │  download.html#abc123/xK9mP2secretkey  ◄── │  ← current (index 2)
        └──────────────────────────────────────────┘

  t=1   File decrypts successfully. SG/Send JS runs:
        history.replaceState(null, '', '/download.html')

        Browser history stack:
        ┌──────────────────────────────────────────┐
        │  [prev page]                             │  ← index 0
        │  [prev page 2]                           │  ← index 1
        │  download.html                        ◄── │  ← current (index 2)
        └──────────────────────────────────────────┘
        ^                                          ^
        The #key is gone from this entry.          The previous entries
        Pressing Back goes to [prev page 2],       are untouched.
        NOT to download.html#key.

  WHAT THIS MEANS:
  ✓  Key is not visible in the URL bar after decrypt
  ✓  Pressing Back does NOT go to the key URL — it was replaced, not pushed
  ✓  No NEW history entry is created (unlike pushState)
  ✗  Pre-decrypt window: key is in URL for ~0.5–1 second during download
  ✗  If you screenshot or share the URL before decrypt, key is captured
  ✗  If browser crashes before replaceState runs, key stays in history

  THE DISTINCTION from pushState (SPA navigation):
  pushState:    [..., page1, page2] + push → [..., page1, page2, newpage]
                Back button → page2
  replaceState: [..., page1, page2] + replace → [..., page1, newpage]
                Back button → page1  (page2 is gone)

  SG/Send uses replaceState, not pushState.
  The key URL entry is REPLACED, not shadowed.
```

Upload Paths (Small vs Large Files)



SECTION 2 — Coming Soon: PKI Layer

Why Symmetric Keys Have a Ceiling

```
CURRENT PROBLEM WITH SYMMETRIC MODEL
═══════════════════════════════════════════════════════════════

  Sender                                Recipient
  ──────                                ─────────

  Generates random AES key K
          │
          │  ←──────────── THE PROBLEM ──────────────────────
          │  K must travel from Sender to Recipient somehow.
          │  Today it travels in the URL fragment.
          │  Whoever gets the URL gets the key.
          │  Whoever has the key can ALSO ENCRYPT.
```

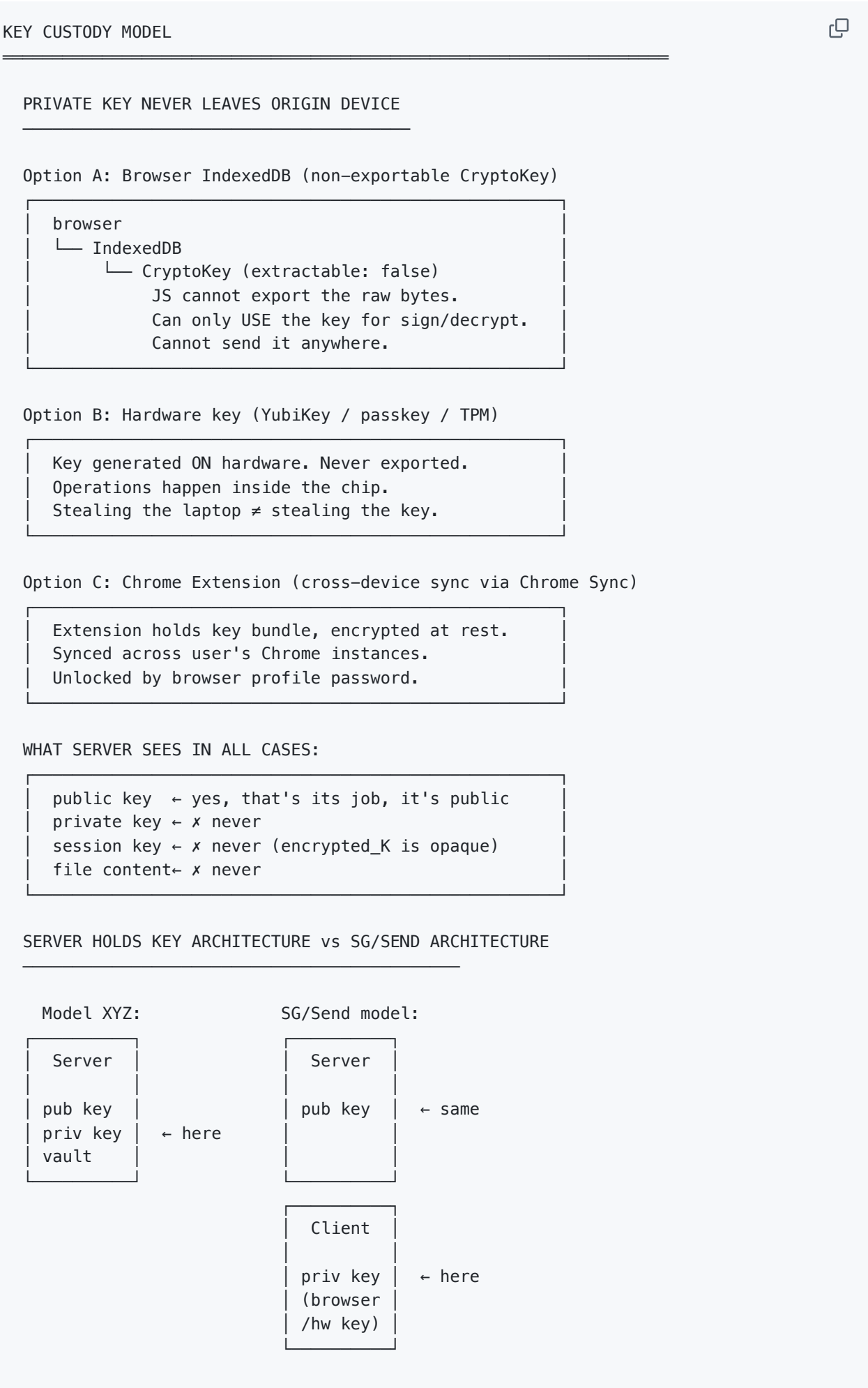
```
          │  ──────────────────────────────────────────────────
          │
          ▼
  URL: download.html#transferId/K_base64
          │
          └──────────────────────────────────────► Recipient opens URL
                                                  Recipient decrypts with K
                                                  ✓ works
                                                  ✗ but K is now "shared"
                                                  ✗ K in transit can be
intercepted
                                                  ✗ K in browser history
(mitigated)
                                                  ✗ anyone with K can re-
encrypt
                                                    malicious file to same
transfer_id
```

The PKI Solution

```
PKI MODEL — HOW IT CHANGES EVERYTHING
═══════════════════════════════════════════════════════════════════

  SETUP (one time per user, done in browser)
  ──────────────────────────────────────────

  Recipient generates key pair in browser:
  ┌──────────────────────────────────────────────────────────────┐
  │  crypto.subtle.generateKey(ECDH P-256)                       │
  │                                                              │
  │  private_key  ← NEVER LEAVES BROWSER                         │
  │                 stored in: IndexedDB (non-exportable)        │
  │                          / hardware key / Chrome extension   │
  │                                                              │
  │  public_key   ← published to SG/Send key registry            │
  │                 anyone can look it up by recipient handle    │
  └──────────────────────────────────────────────────────────────┘

  SG/SEND KEY REGISTRY (server — public data only)
  ┌──────────────────────┬──────────────────────────────────────┐
  │  handle              │  public_key (base64)                 │
  ├──────────────────────┼──────────────────────────────────────┤
  │  dinis               │  MFkwEwYHKoZIzj0CAQY...              │
  │  john                │  MFkwEwYHKoZIzj0CAQY...              │
  │  paul                │  MFkwEwYHKoZIzj0CAQY...              │
  └──────────────────────┴──────────────────────────────────────┘
  Server stores: only public keys. Zero secrets.

═══════════════════════════════════════════════════════════════════

  SEND FLOW WITH PKI
  ──────────────────────────────────────────────────────────────────

  SENDER BROWSER                          RECIPIENT BROWSER
  ──────────────                          ─────────────────

  ┌──────────────┐
```

```
  │  File        │
  └──────┬───────┘
         │
         ▼
  1. Generate session key K (random AES-256)

  2. Look up recipient's public key Pub_R
     GET /keys/ringo → Pub_R

  3. Encrypt file with K
     ciphertext = AES-256-GCM(file, K)

  4. Encrypt K with recipient's public key    ← THE KEY MOMENT
     encrypted_K = ECDH-derive + AES-wrap(K, Pub_R)

     Only Priv_R (which never left recipient's browser)
     can unwrap encrypted_K.
     Sender does NOT know K after this step — it was
     encrypted FOR the recipient specifically.

  5. Sign the ciphertext with sender's private key
     signature = ECDSA(ciphertext, Priv_S)

  6. Upload to server:
     ┌──────────────────────────────────┐
     │  ciphertext    (blob)            │
     │  encrypted_K   (tiny)            │
     │  signature     (tiny)            │
     │  sender_pubkey_ref               │
     └──────────────────────────────────┘
     Server stores all of this. Still sees nothing useful.

  7. Share: https://send.sgraph.ai/download.html#transferId
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
            NO KEY IN URL. Nothing to steal.

                                          8. Download: GET /transfers/{id}
                                             → receive ciphertext, encrypted_K,
signature

                                          9. Unwrap session key:
                                             K = ECDH-unwrap(encrypted_K,
Priv_R)
                                             Priv_R never left this browser.
                                             K is recovered locally.

                                         10. Verify signature:
                                             valid = ECDSA.verify(ciphertext,
signature, Pub_S)
                                             If invalid → STOP. File was
tampered.
                                             If valid → file came from claimed
sender, unmodified.

                                         11. Decrypt:
                                             file = AES-256-GCM-
decrypt(ciphertext, K)
```

PKI Trust Chain — Answering "Who Holds Private Keys?"



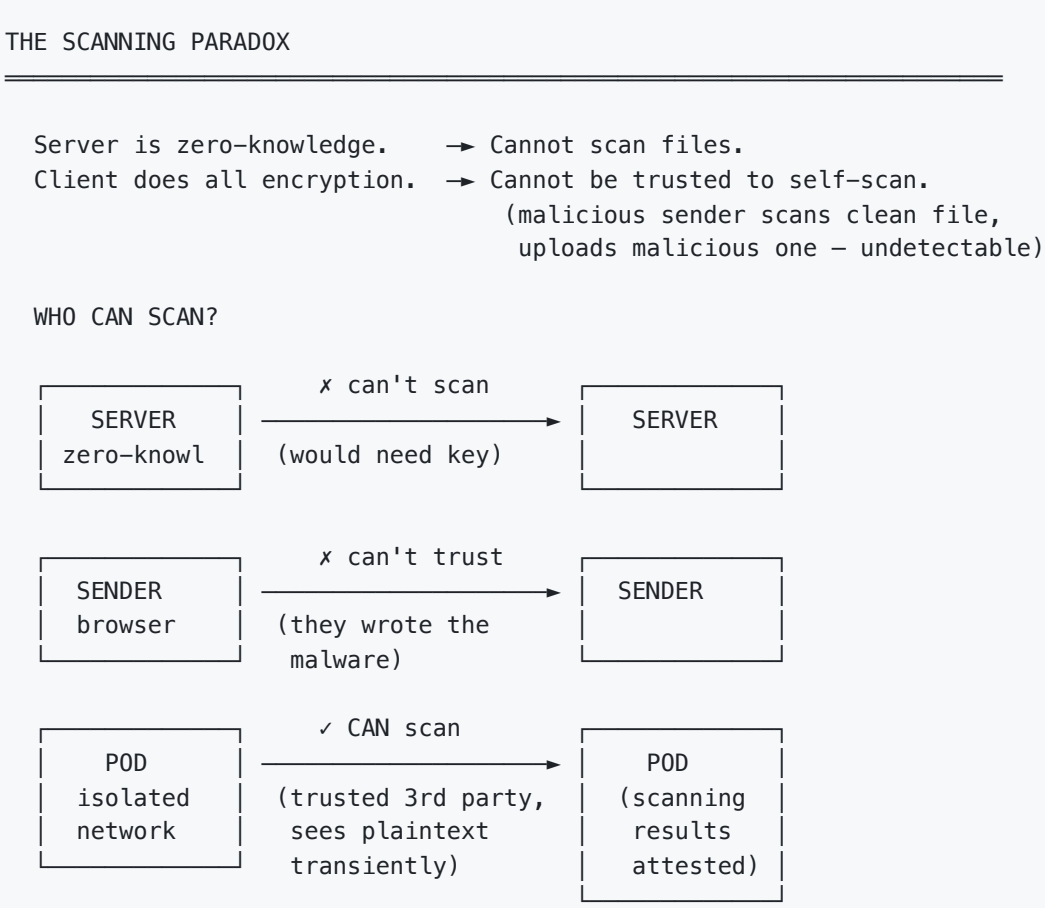
```
  Model XYZ's model: server can decrypt any file at any time.
  SG/Send model: server cannot decrypt any file at any time.
                 This is the differentiator. One breach ≠ all files exposed.
```

Bidirectional Document Workflow

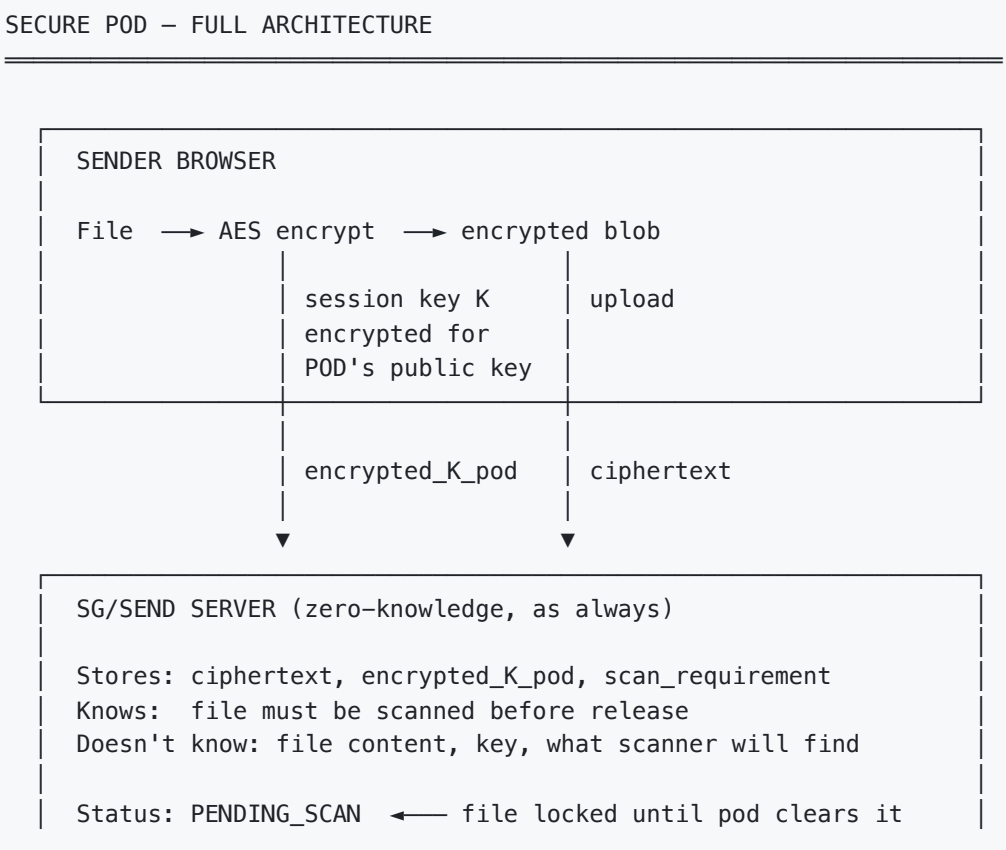
```
"CAN RECIPIENT MODIFY AND SEND BACK?"

  Current answer: Yes — by starting a new transfer in reverse.
  PKI answer: Yes — and provenance is cryptographically proven.

  ──────────────────────────────────────────────────────────────────

  DINIS                          SERVER                        GEORGE
  ─────                          ──────                        ──────

  1. Encrypt "contract_v1.pdf"
     with George's public key
          │
          ▼
  2. Upload → transfer_A ──────────────────────────────────► download
                                                            decrypt with
                                                            George's private
key

  3.                                                        review + edit
                                                            "contract_v2.pdf"

  4.                                                        Encrypt v2
                                                            with Dinis's
public key
                                                                 │
  5. download ◄──────────────────────────── upload → transfer_B ┘
     decrypt with
     Dinis's private key

  6. verify George's signature on v2
     ✓ This file came from George
     ✓ This file was not modified in transit
     ✓ This is the exact bytes George approved

  PROVENANCE CHAIN:
  transfer_A: signed by Dinis, encrypted for George
  transfer_B: signed by George, encrypted for Dinis
  Both are in the audit log. Neither can be forged.
```

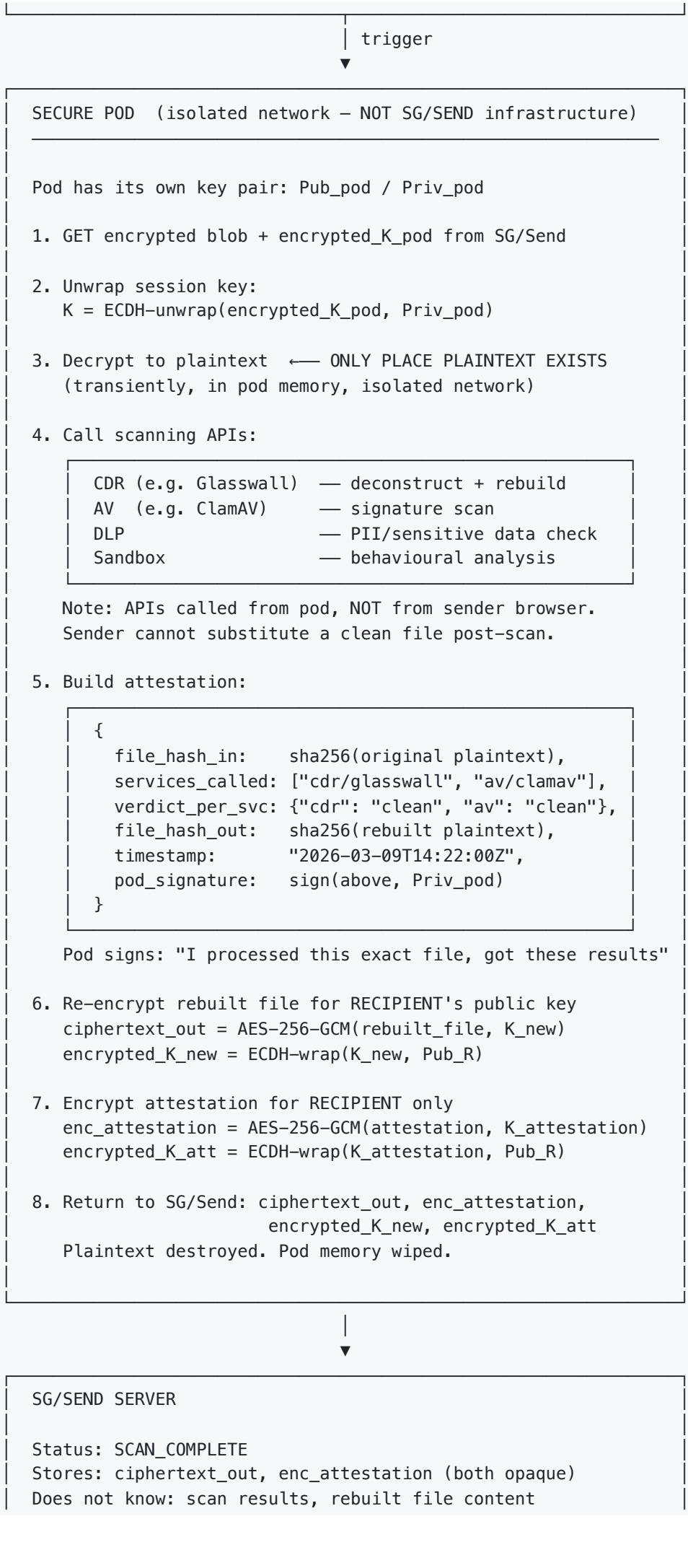
SECTION 3 — Secure Pods: File Analysis Without Knowing the File

The Problem PKI Alone Doesn't Solve



The Secure Pod Architecture

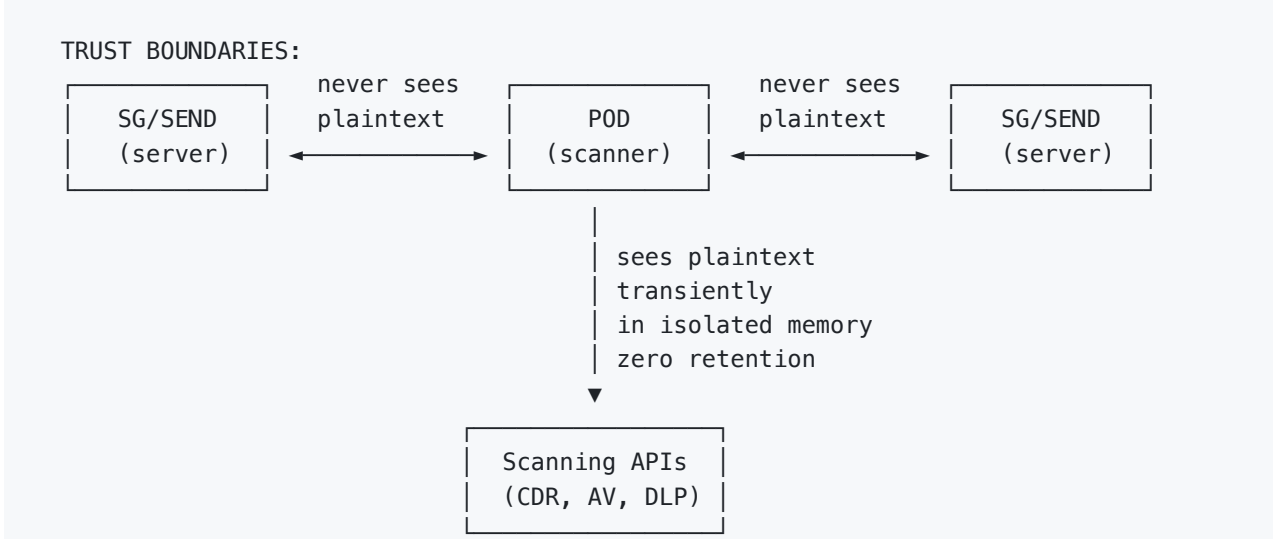



trigger
SECURE POD (isolated network — NOT SG/SEND infrastructure)
Pod has its own key pair: Pub_pod / Priv_pod
1. GET encrypted blob + encrypted_K_pod from SG/Send
2. Unwrap session key:
K = ECDH-unwrap(encrypted_K_pod, Priv_pod)
3. Decrypt to plaintext ←— ONLY PLACE PLAINTEXT EXISTS
(transiently, in pod memory, isolated network)
4. Call scanning APIs:
CDR (e.g. Glasswall) — deconstruct + rebuild
AV (e.g. ClamAV) — signature scan
DLP — PII/sensitive data check
Sandbox — behavioural analysis
Note: APIs called from pod, NOT from sender browser.
Sender cannot substitute a clean file post-scan.
5. Build attestation:
{
file_hash_in: sha256(original plaintext),
services_called: ["cdr/glasswall", "av/clamav"],
verdict_per_svc: {"cdr": "clean", "av": "clean"},
file_hash_out: sha256(rebuilt plaintext),
timestamp: "2026-03-09T14:22:00Z",
pod_signature: sign(above, Priv_pod)
}
Pod signs: "I processed this exact file, got these results"
6. Re-encrypt rebuilt file for RECIPIENT's public key
ciphertext_out = AES-256-GCM(rebuilt_file, K_new)
encrypted_K_new = ECDH-wrap(K_new, Pub_R)
7. Encrypt attestation for RECIPIENT only
enc_attestation = AES-256-GCM(attestation, K_attestation)
encrypted_K_att = ECDH-wrap(K_attestation, Pub_R)
8. Return to SG/Send: ciphertext_out, enc_attestation,
encrypted_K_new, encrypted_K_att
Plaintext destroyed. Pod memory wiped.
SG/SEND SERVER
Status: SCAN_COMPLETE
Stores: ciphertext_out, enc_attestation (both opaque)
Does not know: scan results, rebuilt file content

```
┌──────────────────────────────────────────────────────────────────┐
└──────────────────────────────────┬───────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────┐
│  RECIPIENT BROWSER                                               │
│                                                                  │
│  Download ciphertext_out + enc_attestation                       │
│       │                                                          │
│       ▼                                                          │
│  Unwrap K_new with Priv_R → decrypt rebuilt file                 │
│       │                                                          │
│       ▼                                                          │
│  Unwrap K_att with Priv_R → decrypt attestation                  │
│       │                                                          │
│       ▼                                                          │
│  Verify pod signature on attestation (Pub_pod is published)      │
│  Verify hash_out matches decrypted file                          │
│       │                                                          │
│       ▼                                                          │
│  ✓ "This file was rebuilt by Glasswall + scanned by ClamAV"      │
│  ✓ "The file I'm opening is byte-for-byte what the pod rebuilt"  │
│  ✓ "No substitution happened between scan and delivery"          │
└──────────────────────────────────────────────────────────────────┘
```

Why the Pod Must Be a Third Party

```
WHO SHOULD RUN THE POD?
═══════════════════════════════════════════════════════════════════

  Option 1: SG/Send runs the pod

  ┌──────────────────────────────────────────────────────────┐
  │  ✗ SG/Send would need to handle plaintext                │
  │  ✗ Violates the zero-knowledge guarantee                 │
  │  ✗ SG/Send becomes a target for subpoenas                │
  │  ✗ The entire security model collapses                   │
  └──────────────────────────────────────────────────────────┘

  Option 2: Scanning company runs the pod (Glasswall, OPSWAT, etc.)

  ┌──────────────────────────────────────────────────────────┐
  │  ✓ Scanning company sees plaintext — but that's their job  │
  │  ✓ SG/Send still never sees plaintext                    │
  │  ✓ Scanning company is contractually bound:              │
  │      - zero data retention after scan                    │
  │      - cannot share results except with attested recipient │
  │  ✓ SG/Send's zero-knowledge guarantee holds              │
  │  ✓ Scanning company gets a revenue stream per scan       │
  │  ✓ Two separate liability domains                        │
  └──────────────────────────────────────────────────────────┘

  Option 3: Receiver's organisation runs its own pod

  ┌──────────────────────────────────────────────────────────┐
  │  ✓ Government / defence use case                         │
  │  ✓ Air-gapped pod, no external API calls                 │
  │  ✓ Scanning engine embedded in pod runtime               │
  │  ✓ "Cross-domain solution" — files cross from untrusted  │
  │    to trusted domain, scanned at the boundary            │
  └──────────────────────────────────────────────────────────┘
```



Liability and Charging

```
WHO'S RESPONSIBLE IF A VIRUS GETS THROUGH?
═══════════════════════════════════════════════════════════════

  SCENARIO A: Sender does not request scanning (basic tier)
  ──────────────────────────────────────────────────────────
  Recipient downloads → gets virus
  SG/Send liability: ✗ none — courier is not responsible for package contents
                           Recipient chose no scanning
  Analogy: Royal Mail does not open your letters to check for anthrax

  SCENARIO B: Sender requests scanning, scanner misses it
  ──────────────────────────────────────────────────────────
  SG/Send liability: ✗ none — scanning done by certified third party
  Scanner liability: contract-defined, scanner carries professional indemnity
  SG/Send is the orchestrator, not the scanner

  SCENARIO C: Receiver mandates scanning, sender bypasses it
  ──────────────────────────────────────────────────────────
  SG/Send blocks delivery until scan is complete.
  Cannot be bypassed — scan is enforced by platform, not by user honesty.

  CHARGING MODEL:
  ┌──────────────────────────────────────────────────────────┐
  │                                                          │
  │  Sender uploads     →  scan triggered                    │
  │  Scan costs X       →  charged to sender's token balance │
  │                                                          │
  │  Receiver mandates  →  "This recipient requires CDR + AV"│
  │  higher tier        →  Sender sees cost before uploading │
  │                     →  Sender can accept or abandon      │
  │                                                          │
  │  Basic (no scan)    →  £0.01/transfer (current)          │
  │  AV scan            →  £0.01 + scan API cost             │
  │  CDR + AV           →  £0.01 + CDR cost + AV cost        │
  │  CDR + AV + DLP     →  £0.01 + all scan costs            │
  │                                                          │
  │  Scanning companies earn per API call.                   │
  │  SG/Send earns per transfer + optional margin on scans.  │
```

```
  │  Receiver gets security without procurement overhead.      │
  └────────────────────────────────────────────────────────────┘
```

Summary Comparison

```
CAPABILITY MATRIX
═══════════════════════════════════════════════════════════════════════

  Feature                          │ Today  │ PKI    │ + Pods
  ─────────────────────────────────┼────────┼────────┼────────
  Server sees plaintext            │  ✗     │  ✗     │  ✗
  Server holds private keys        │  ✗     │  ✗     │  ✗
  Key in URL                       │  ✓     │  ✗     │  ✗
  Recipient-specific encrypt       │  ✗     │  ✓     │  ✓
  Sender provenance (signed)       │  ✗     │  ✓     │  ✓
  Post-scan substitution blocked   │  ✗     │  ✗     │  ✓
  Malware scanning                 │  ✗     │  ✗     │  ✓
  CDR / file rebuild               │  ✗     │  ✗     │  ✓
  DLP / PII detection              │  ✗     │  ✗     │  ✓
  Cryptographic scan attestation   │  ✗     │  ✗     │  ✓
  Bidirectional signed exchange    │  ✗     │  ✓     │  ✓
  Files up to 1 GB (soon 5TB)      │  ✓     │  ✓     │  ✓
  Zero-knowledge guarantee         │  ✓     │  ✓     │  ✓

═══════════════════════════════════════════════════════════════════════

  WHAT NEVER CHANGES:
  The server cannot read your files.
  That constraint is load-bearing. Everything is designed around it.
```

SG/Send is open source (Apache 2.0) — https://github.com/the-cyber-boardroom/SGraph-AI__App__Send
Zero-knowledge encrypted file sharing — https://send.sgraph.ai
Secure pod architecture brief — on request